BOUPASSIA Chris
COLLINOT Lise
MAAREK Joshua

RAPPORT – Utilisation d'IA pour le TP Neo4j SNCF

1. Contexte

Dans ce TP, l'objectif est d'utiliser Neo4j pour modéliser et exploiter un réseau de trajets ferroviaires à partir d'un dataset fourni par la SNCF.

Cependant, certaines étapes nécessaires pour ce projet ne sont pas intuitives et/ou écrites dans les codes qui nous ont été soumis dans le cours. Nous avons donc utilisé de l'IA générative pour nous aider sur certaines parties du code final.

Notre projet comporte deux parties principales :

- un script Python permettant de **préparer et transformer les données du dataset**
- un script Python permettant **d'importer ces données dans Neo4j et d'interroger la base**

Certaines parties de ces scripts ont été réalisées avec l'aide d'outils d'IA générative.

2. Parties du code où l'IA nous a aidés pour le dump

2.1 Préparation des données avec Pandas

Le notebook du cours n'explique pas comment préparer le dataset SNCF avant son import dans Neo4j.

Nous avons utilisé l'IA pour nous aider à structurer les données avec **pandas** et sélectionner les colonnes utiles du dataset :

```
df = df[[

    "DATE",
    "TRAIN_NO",
    "Origine",
    "Origine IATA",
    "Destination",
    "Destination IATA",
    "Heure_depart",
    "Heure_arrivee"

]]
```

L'IA nous a aidés à identifier les colonnes pertinentes pour la modélisation du graphe.

2.2 Création des nœuds (gares)

Pour construire les nœuds représentant les gares, nous avons utilisé **pandas** afin d'extraire toutes les gares d'origine et de destination puis de supprimer les doublons.

```
gares = pd.concat([
    df[["Origine IATA","Origine"]].rename(
        columns={"Origine IATA":"iata","Origine":"nom"}
    ),
    df[["Destination IATA","Destination"]].rename(
        columns={"Destination IATA":"iata","Destination":"nom"}
    )
]).drop_duplicates()
```

L'IA nous a aidés à trouver une manière simple d'obtenir la liste des gares uniques.Ici pd.concat permet de fusionner plusieurs dataframes en une seule, ce dont on a besoin pour optimiser « Origine IATA » et « Destination IATA »

2.3 Création des segments de trajets

Nous avons également créé un fichier contenant les **segments de trajets**, c'est-à-dire les trajets directs entre deux gares successives.

```
segments = df[[
    "TRAIN_NO",
    "DATE",
    "Origine IATA",
    "Destination IATA",
    "Heure_depart",
    "Heure_arrivee"
]].drop_duplicates()
```

Ces segments sont ensuite exportés sous forme de fichier CSV afin de faciliter l'import dans Neo4j :

```
segments.to_csv("segments_relations.csv", index=False)
```

Cette étape de transformation des données n'était pas décrite dans le cours et a été réalisée avec l'IA.

2.4 *Reconstruction des trajets complets avec `groupby`*

Le dataset contient plusieurs lignes pour un même train.
Pour reconstruire un trajet complet (origine → destination), nous avons utilisé la fonction **groupby** de pandas :

```
trajets = df.groupby(["TRAIN_NO","DATE"]).agg(
    origine=("Origine","first"),
    destination=("Destination","last"),
    depart=("Heure_depart","first"),
    arrivee=("Heure_arrivee","last")
).reset_index()
```

Cette opération permet d'obtenir un trajet complet pour chaque train et chaque date.

Cette partie n'était pas expliquée dans le cours et l'IA nous a aidés à trouver cette solution.

2.5 *Import en batch dans Neo4j avec* `UNWIND`

Le notebook du cours montre la création de nœuds et relations simples, mais ne traite pas l'import d'un dataset de plusieurs milliers de lignes.

L'IA nous a proposé d'utiliser **UNWIND** pour importer les données plus efficacement :

```
UNWIND $rows AS row
MERGE (g:Gare {iata:row.iata})
SET g.nom=row.nom
```

et pour les relations :

```
CREATE (o)-[:TRAJET {

    train_no:row.train_no,
    date:row.date,
    heure_depart:row.heure_depart,
    heure_arrivee:row.heure_arrivee

}]->(d)
```

Cela permet d'insérer plusieurs milliers de lignes de données rapidement dans Neo4j.

BOUPASSIA Chris
COLLINOT Lise
MAAREK Joshua

3. Site Web

3.1 Parsing

Nous avons utilisé l'IA pour nous aider à concevoir et corriger la fonction de parsing du fichier CSV. Cette fonction est essentielle car elle permet de lire correctement les données contenues dans les fichiers CSV (trajets_complets.csv et gares_nodes.csv). L'IA nous a aidés à comprendre comment gérer les séparateurs, les guillemets et les différentes colonnes afin d'éviter les erreurs de lecture des données. Grâce à cette aide, nous avons pu créer un parser plus robuste capable de traiter correctement les lignes du fichier CSV.

```
function parseCSVLine(line){
let result = [];
let current = '';
let insideQuotes = false;
for(let i=0;i<line.length;i++){

let char=line[i];

if(char === '"'){
insideQuotes = !insideQuotes;
}

else if(char === ',' && !insideQuotes){
result.push(current.replace(/^"|"$/g,'').trim());
current='';
}

else{
current += char;
}

}

result.push(current.replace(/^"|"$/g,'').trim());

return result;
```

BOUPASSIA Chris
COLLINOT Lise
MAAREK Joshua

3.2 Affichage des trajets

L'IA nous a également aidés à structurer la fonction permettant d'afficher les trajets dans le tableau HTML. Elle nous a expliqué comment parcourir la liste des trajets et générer dynamiquement les lignes du tableau avec JavaScript. Cette aide nous a permis de mieux comprendre la manipulation du DOM et la création dynamique d'éléments HTML afin d'afficher les informations comme la date, le numéro de train, la gare de départ et d'arrivée ainsi que les horaires.

```
function showTrips(list){

let tbody = document.getElementById("tableBody");

tbody.innerHTML = "";

list.forEach(t=>{

let tr = document.createElement("tr");

tr.innerHTML = `
<td>${t.date}</td>
<td>${t.train_no}</td>
<td>${t.origine}</td>
<td>${t.heure_depart}</td>
<td>${t.destination}</td>
<td>${t.heure_arrivee}</td>
`;

tbody.appendChild(tr);

});

document.getElementById("shownResults").innerText = list.length;

}
```

3.3 Réinitialisation des trajets

La fonction de réinitialisation permet de remettre les champs de recherche à zéro et d'afficher de nouveau l'ensemble des trajets. L'IA nous a aidés à structurer cette fonction afin qu'elle vide les champs de formulaire et recharge la liste complète des trajets. Cela améliore l'expérience utilisateur car il est possible de revenir facilement à la liste complète des résultats.

```
/* reset */

function showAllTrips(){

document.getElementById("depart").value="";
document.getElementById("arrivee").value="";
document.getElementById("date").value="";

showTrips(trips);

}
```

BOUPASSIA Chris
COLLINOT Lise
MAAREK Joshua

3.4 Initialisation

L'IA nous a aidés à organiser la phase d'initialisation de l'application. Cette étape consiste à charger les données depuis les fichiers CSV (gares et trajets) lors du lancement de la page. Grâce à l'IA, nous avons compris comment utiliser les fonctions asynchrones (async/await) pour charger les données dans le bon ordre et éviter les erreurs d'affichage. L'initialisation permet ainsi de préparer correctement

```
async function init(){

await loadStations();
await loadTrips();

showTrips(trips);

document.getElementById("message").innerText="Données chargées";

}

init();
```

4. Apprentissage personnel

L'utilisation de l'IA nous a permis de :

- Mieux comprendre comment préparer un dataset avant de l'importer dans Neo4j,
- Découvrir l'utilisation de **pandas** pour transformer les données,
- Apprendre à structurer un graphe représentant un réseau de transport,
- Comprendre l'utilisation de **UNWIND** pour importer efficacement un grand volume de données dans Neo4j.
- Comprendre ce qu'est un « parser »
- Manipuler les DOM
- Utiliser les fonctions asynchrones